



Detector Calibration

Data Science – NPAC

Binh T. Y. Nguyen, Waddia Summan, Theodora Kaskitsi

December 19, 2025

Problem Setup

1 Introduction



Calibration of a small, telescope-like multilayer detector using 1 GeV electrons originating from the point $(0, 0, 0)$.

The detector is made of 7 active sensor layers (indexing starts at 0).

Final objective: characterise a newly built sensor that will later be integrated into a larger particle physics experiment.

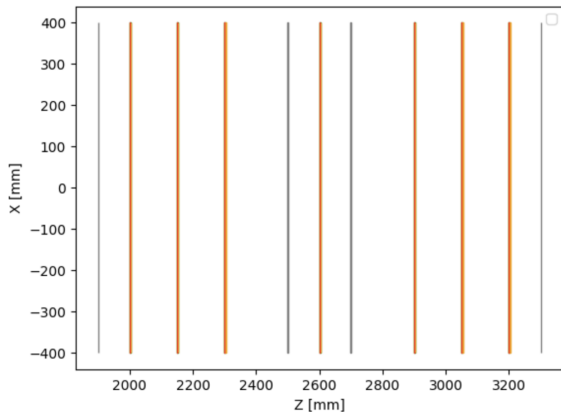


Figure 1: Position of the detector layers in the Z-X plane (in mm). Coloured layers represent active sensors.

Detector Calibration

1 Introduction



- This work focuses on the first stage of detector characterisation: **calibration**.
- The objective is to determine the **particle crossing position** in each detector layer using the measured energy deposition.
- Calibration is performed independently for each layer and provides the spatial calibration parameters:

$$(\mu_x, \mu_y, \sigma_x, \sigma_y)$$

- These parameters describe the hit position and spatial spread of the detector response and are later used for particle tracking.

Methods

2 Methodology



Figure 2: Overall workflow

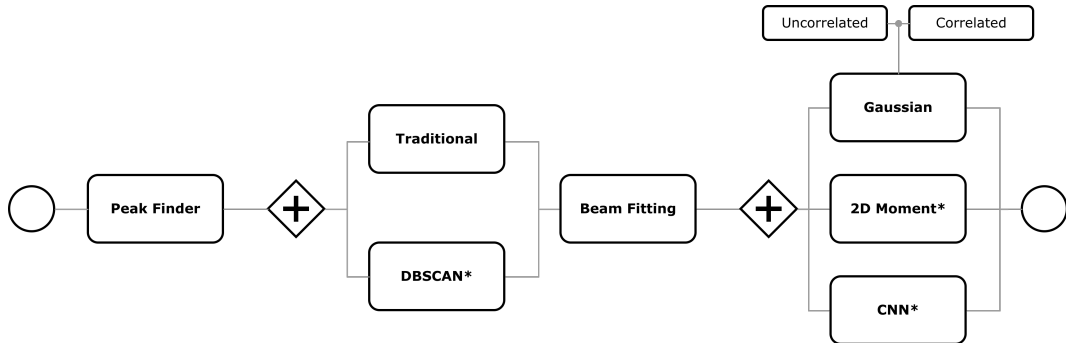


Figure 3: Work flow for peak finder and beam fitting methods

Data Representation

2 Methodology



Throughout this analysis, we focus on the calibration of a single detection event (event 47) from a 1 GeV electron shower, processed independently for each detector layer.

- The detector response is provided as digitised hit data stored in CSV format.
- For a given layer, hit information can be represented either as:
 - a **scatter plot** of pixel hit positions, or
 - a **2D energy image (matrix)** where deposited energy is accumulated per pixel.

Data Representation

2 Methodology

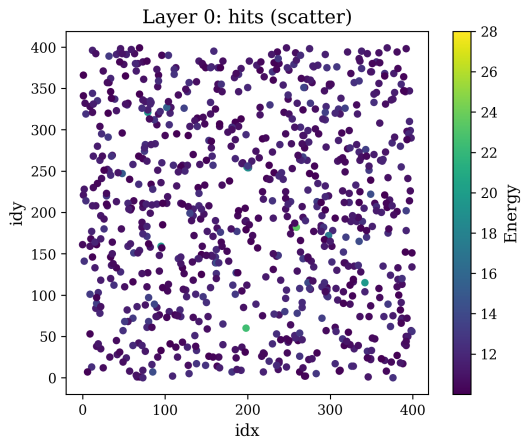


Figure 4: Scatter plot of hit positions.

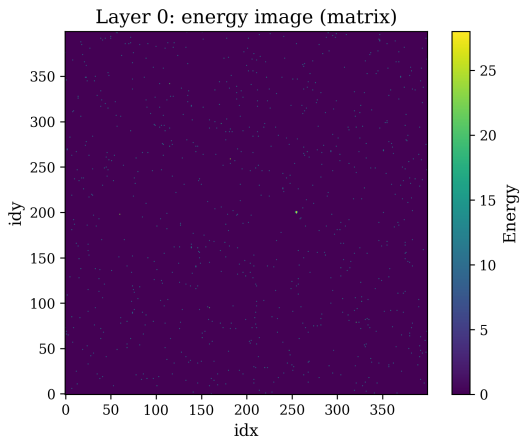


Figure 5: 2D energy image (matrix) representation.

Noise Separation and Peak Finder

2 Methodology



- Pixel energies are dominated by background noise.
- For each detector layer, the noise distribution is approximated by a **Gaussian model** centered on $\mu_0 = \min(E)$.
- A **threshold** $\mathcal{T} = \mu_0 + 3\sigma_{\text{noise}}$ is defined to suppress noise fluctuations.
- Pixels above this threshold and larger or equal than their neighbours are treated as signal candidates (**peaks**).

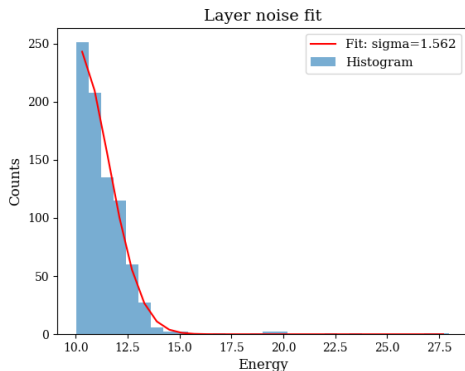
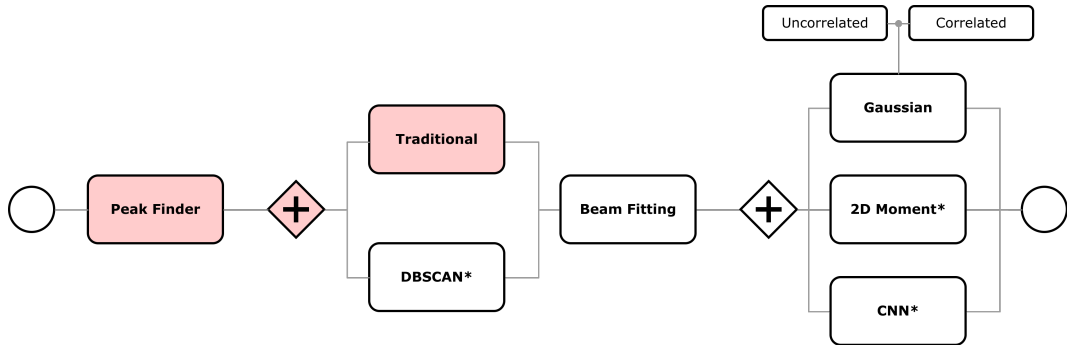


Figure 6: Gaussian noise fit for layer 0.

Traditional Method

2 Methodology



Extended Peaks: True Beam Crossings

2 Methodology



- Peaks above $3\sigma_{\text{noise}}$ include many false positives (isolated high-energy pixels).
- True beam crossings deposit energy over **multiple neighbouring pixels**.
- A peak is classified as an **extended peak** if at least 3 neighbouring pixels have non-zero energy.

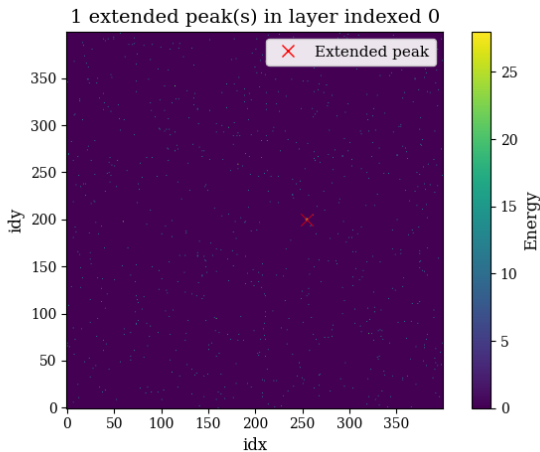
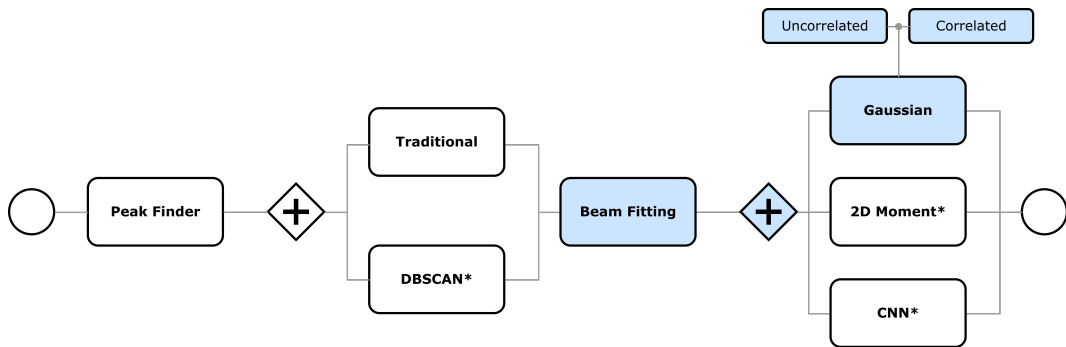


Figure 7: Extended peaks identified in a detector layer.

Beam Characterisation with 2D Gaussian Models

2 Methodology



Beam Characterisation with 2D Gaussian Models

2 Methodology



- To characterise beam crossings, each extended peak is modeled with a **2D Gaussian distribution**.
- The goal is to estimate the beam center and spatial widths:

$$(\mu_x, \mu_y, \sigma_x, \sigma_y).$$

- We first apply an **uncorrelated 2D Gaussian**, assuming independent spreads along the x and y directions.
- For peaks with tilted or elongated shapes, this assumption breaks down.
- A **correlated 2D Gaussian** is then used to account for correlations between x and y , improving the beam description.

Uncorrelated vs Correlated Gaussian Fits

2 Methodology

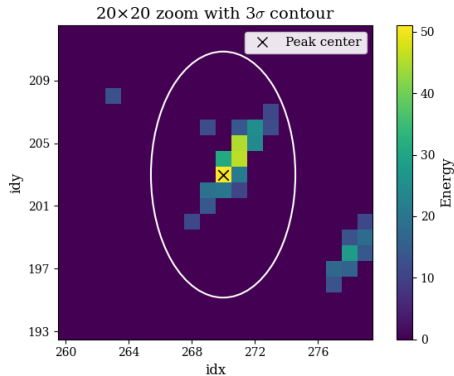


Figure 8: Uncorrelated 2D Gaussian fit (layer 3).

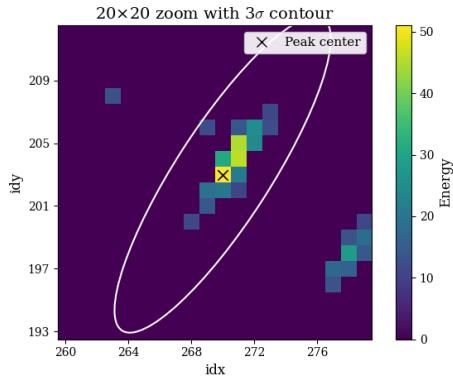


Figure 9: Correlated 2D Gaussian fit (layer 3).

Uncorrelated vs Correlated Gaussian Fits

2 Methodology

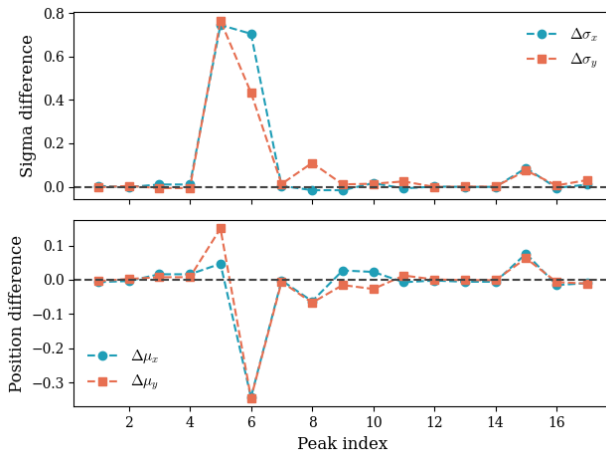


Figure 10: Comparison of calibration parameters obtained with uncorrelated and correlated 2D Gaussian fits for the same set of peaks.

▶ Introduction

▶ Methodology

▶ **Extensions**

▶ Parallelization

▶ Conclusions and Outlook

▶ Additional slides

Section 3.1

Motivation for extensions

Double-Peak Ambiguities

3 Extensions

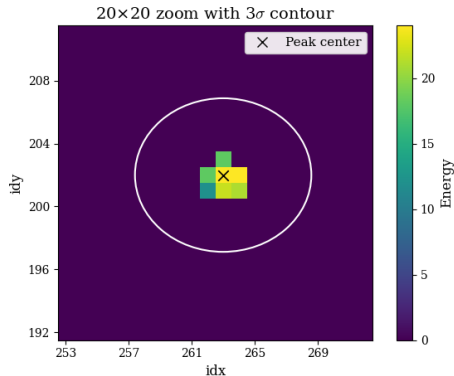


Figure 11: Extended peak found in layer 2.

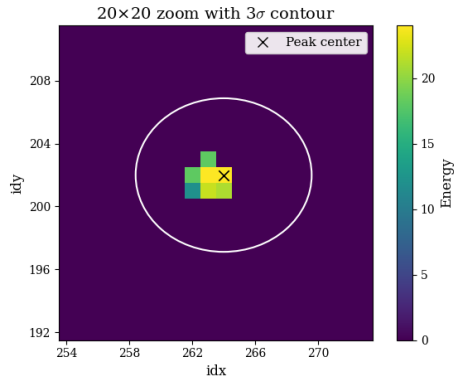


Figure 12: Second peak found in the same region.

Profiling: Computational Limitations

3 Extensions



- **Profiling** shows that the 2D Gaussian fits of the extended peaks dominate the computation time.
- This motivates the exploration of faster, scalable alternatives.

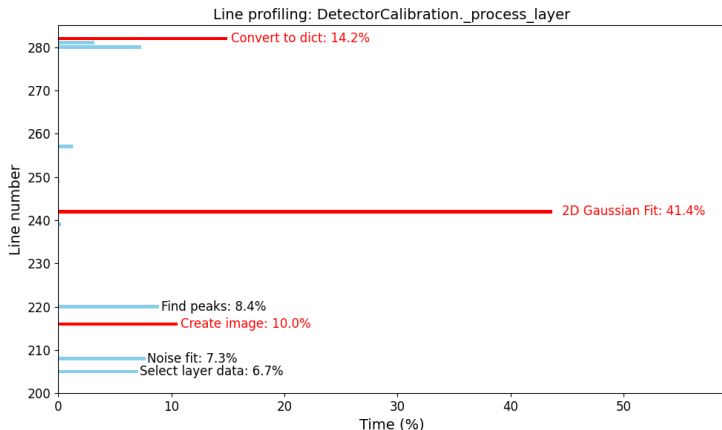


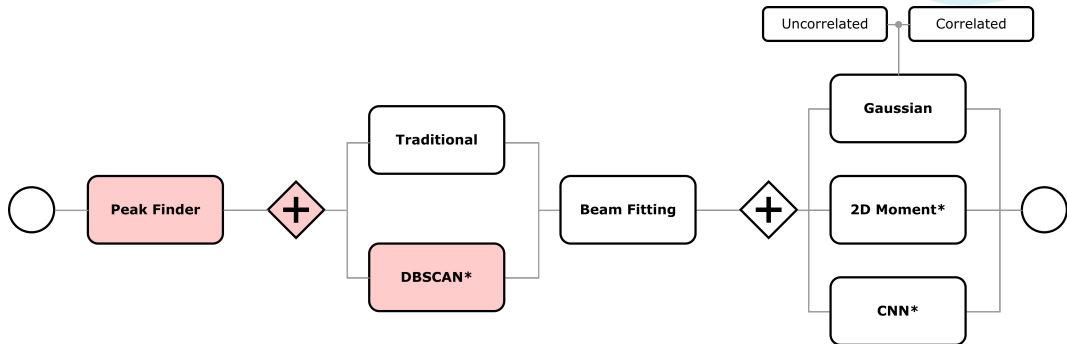
Figure 13: Line profiling highlighting the computational bottleneck of 2D Gaussian fitting.

Section 3.2

Extensions

DBSCAN for double-peak problem

3 Extensions



DBSCAN

3 Extensions

Concept: DBSCAN groups points in high-density regions and marks isolated points as noise.



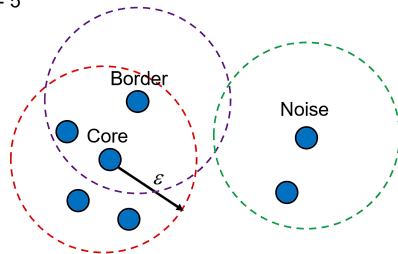
- **Parameters:** ϵ (neighborhood radius), `min_samples` (min points for a core)
- **Point types:** Core ($\geq \text{min_samples}$ neighbors), Border (near a core but $< \text{min_samples}$), Noise
- Clusters are formed by connected cores; border points join nearby cores

Implementation: DBSCAN from `sklearn.cluster`. Inputs are coordinates of pixels above a threshold.

- `eps` = 1.5
- `min_samples` = 3

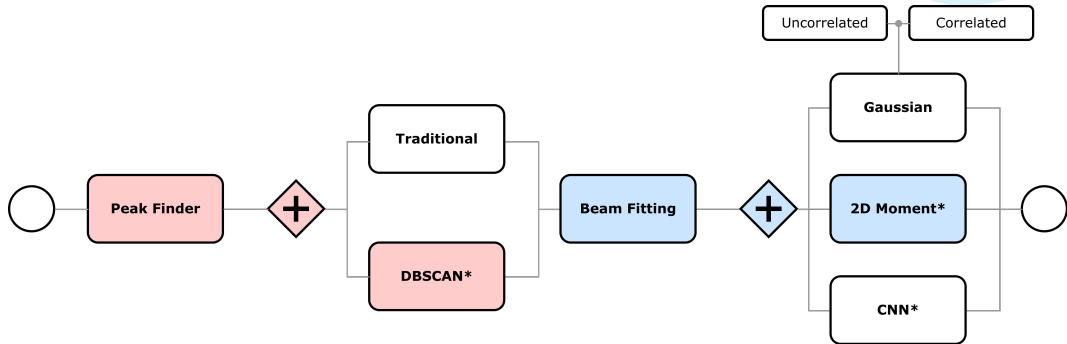
Clusters form when ≥ 3 energetic pixels lie within a 2-pixel radius.

MinPts = 5



2D Moments

3 Extensions



2D Moments

3 Extensions

In image analysis, moments are intensity-weighted averages that describe the position and shape of an image peak. For a 2D image $I(x, y)$, the raw moments are defined as:

$$M_{pq} = \sum_x \sum_y x^p y^q I(x, y)$$

- **Zeroth moment (total intensity):**

$$M_{00} = \sum_x \sum_y I(x, y)$$

- **First moments (centroid):**

$$\mu_x = \frac{M_{10}}{M_{00}}, \quad \mu_y = \frac{M_{01}}{M_{00}}$$

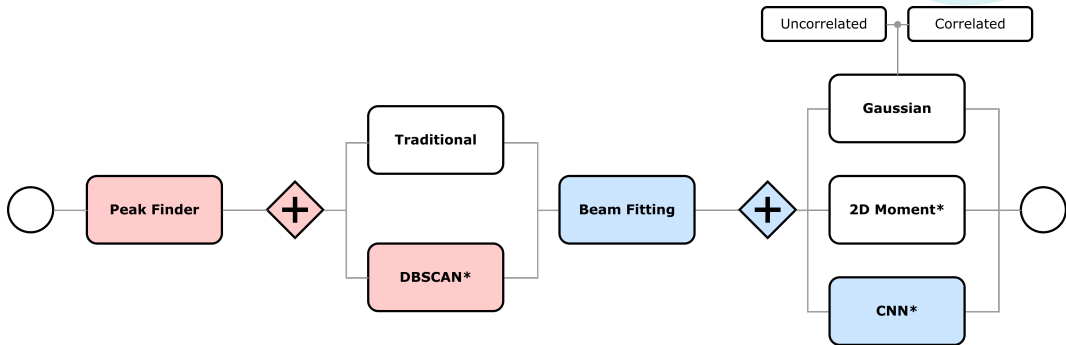
- **Second central moments (spread):**

$$\sigma_x^2 = \frac{\sum (x - \mu_x)^2 I(x, y)}{M_{00}}, \quad \sigma_y^2 = \frac{\sum (y - \mu_y)^2 I(x, y)}{M_{00}}$$

These quantities provide a direct estimate of the peak position (μ_x, μ_y) and spatial extent (σ_x, σ_y) without assuming a specific functional form.

Convolutional Neural Network

3 Extensions



CNN

3 Extensions

Concept: A CNN is designed for grid-like data (images). It learns hierarchical features, from local patterns to global structures.

- **Convolution:** Learn local features using sliding kernels

$$G[m, n] = (I * K)[m, n] = \sum_j \sum_k I[m - j, n - k] K[j, k]$$

where I is the image and K a learnable filter.

- **Activation (ReLU):** Introduces non-linearity

$$f(x) = \max(0, x)$$

- **Fully Connected Layers:** Perform regression on extracted features

$$y = Wx + b$$

Implementation: A CNN is used to directly regress beam parameters

$$(\mu_x, \mu_y, \sigma_x, \sigma_y)$$

from a small image patch around an extended peak. The true values come from a Gaussian fit, and the network is trained using the smooth L1 loss.

Architecture

3 Extensions

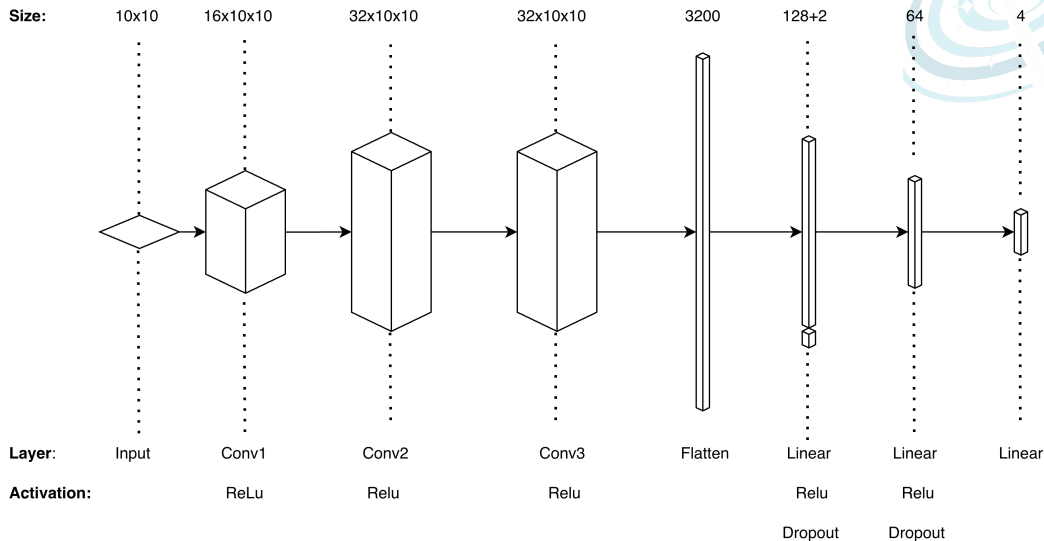


Figure 14: CNN Architecture. **Conv** denotes 3×3 convolutional layers with padding 1, and **Linear** denotes fully connected layers. Inputs are 10×10 image patches. Outputs are mean and log-variance of x and y .

Comparing different beam fitting methods

3 Extensions

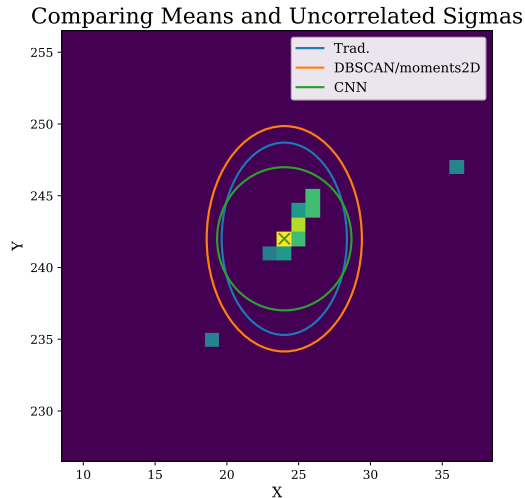


Figure 15: Uncorrelated 2D Gaussian fit

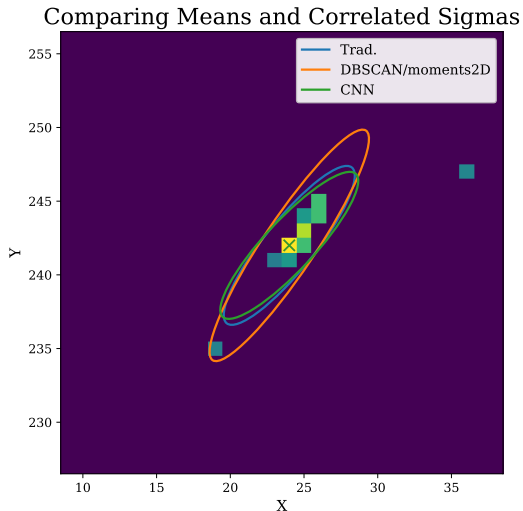


Figure 16: Correlated 2D Gaussian fit

Event-Level Parallel Computation

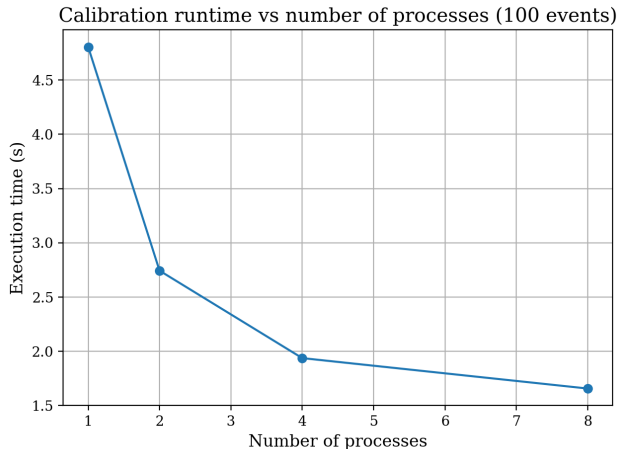
4 Parallelization



- So far, the calibration has been applied to a **single event**.
- In practice, calibration must be performed over **thousands of events**.
- Since events are independent, calibration can be parallelized across events using Python multiprocessing.
- Each process runs the full calibration on a separate event, exploiting multiple CPU cores.
- This corresponds to **task-level, embarrassingly parallel** computation on a single machine.

Parallel Performance and Scaling

4 Parallelization



- Strong scaling is observed up to about **4 processes**.
- Beyond this point, performance saturates because CPU scheduling and memory management overhead outweigh the benefits of additional processes.

Figure 17: Calibration runtime as a function of the number of processes for 100 events.

Conclusions

5 Conclusions and Outlook



Peak finders

- Traditional method produced the double peaks problem.
- DBSCAN resolved the double peaks problem.

Beam fitting

- Uncorrelated Gaussian fit cannot capture the true shape of the beam.
- Correlated Gaussian fit can capture the shape.
- Gaussian fit is the bottleneck in the module.
- 2D moments and CNN, after training, are much faster.
- 2D moments tend to overestimate the variance compared to Gaussian fit, especially at low resolution. This happens because pixelization smears the intensity, increasing the apparent spread.

Parallelization

- Event-level multiprocessing provides strong speedup up to 4 processes. Beyond that, runtime saturates.

Outlook

5 Conclusions and Outlook



- With more time, further research on CNNs could improve statistics and performance.
- Currently, Gaussian fits are used as a benchmark, but training on simulated data could provide better accuracy, purity, and other metrics.
- Improve test coverage for experimental code modules: DBSCAN and CNN.
- Set up automatic publishing in the CI pipeline on tag creation, although some issues still need to be resolved.

Q&A

Thank you for listening!

Training loss function

6 Additional slides

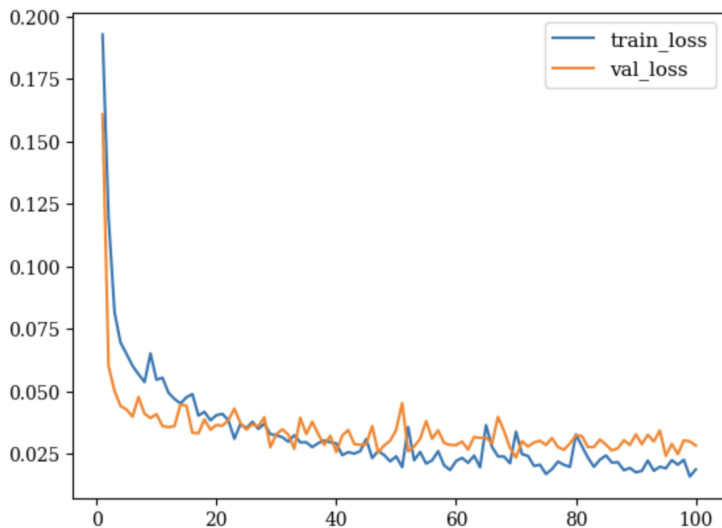


Figure 18: Loss Function